2022 IEEE 28th International Conference on Parallel and Distributed Systems (ICPADS) | 978-1-6654-7315-6/23/$31.00 ©2023 IEEE | DOI: 10.1109/ICPADS56603.2022.00063

# WMDRS: Workload-Aware Performance Model Based Multi-Task Dynamic-Quota Real-Time Scheduling for Neural Processing Units

1st Chong Liu
*School of Computer Science*
*Northwestern Polytechnical University*
Xi'an, China
lchong0258@163.com

2nd Yuan Yao
*School of Computer Science*
*Northwestern Polytechnical University*
Xi'an, China
yaoyuan@nwpu.edu.cn

3rd Yi Dang
*School of Computer Science*
*Northwestern Polytechnical University*
Xi'an, China
dangyi0223@163.com

4th Gang Yang
*School of Computer Science*
*Northwestern Polytechnical University*
Xi'an, China
yeungg@nwpu.edu.cn

5th Wei Jia
*Department of Aviation Electronics*
*Xian Aisheng Technology Group Co.,Ltd*
Xi'an, China
jiawei@nwpu.edu.cn

6th Xinyu Tian
*Department of aviation electronics*
*Xian Aisheng Technology Group Co.,Ltd*
Xi'an, China
windtian2002@163.com

7th Xingshe Zhou
*School of Computer Science*
*Northwestern Polytechnical University*
Xi'an, China
zhouxingshe@nwpu.edu.cn

***Abstract*—To further improve the capacity of airborne embedded system for dealing with deep learning (DL) applications and reduce overall power consumption, it is necessary to equip Neural Processing Units (NPUs). Comparing with the cloud system, the airborne embedded system usually has a fixed application set, but strict real-time constraints. Unfortunately, the inherent NPU scheduler does not consider the application priority, which cannot provide the sufficient real-time capability for the airborne embedded system. At present, there are few researches on multi-task real-time scheduling for NPUs. Therefore, we propose WMDRS, a workload-aware performance model multi-task dynamic-quota real-time scheduling for Neural Processing Units. The NPU performance model based on workload-awareness can accurately predict the remaining execution time of a task, which is running concurrently with other tasks on NPU. The multi-task dynamic-quota real-time scheduling algorithm can provide the approximate preemption by dynamically adjusting NPU computing resources for active applications. In addition, we implement a prototype NPU scheduler without any hardware extension. Furthermore, the proposed NPU performance model and real-time scheduling algorithm are evaluated in realistic application sets. Experimental results demonstrate that WMDRS can achieve low prediction error and high scheduling success ratio.**

***Index Terms*—real-time scheduling, NPU performance model, embedded system, preemptive scheduling**

## I. Introduction

With the improvement concerning the intelligence level of the Unmanned Aerial Vehicle (UAV) and the significant progress made by deep learning (DL) in almost all fields of machine learning (ML), there is a strong demand of computational capacity for airborne embedded system to deal with an increasing number of DL applications. However, the traditional architecture of CPU+GPU cannot meet the needs of deep learning tasks well. To meet the demands of computation-hungry deep neural network (DNN) based machine learning algorithms, researchers have put enormous efforts into developing DNN accelerators [1] [2] [3] [4], also known as neural processing units (NPUs). As a special embedded neural network chip mainly using ASIC technology, the NPU is featured with powerful computing performance and has been applied to end-side AI inference computing and some applications on the cloud side. Apart from that, compared with the CPU and the GPU, the NPU is driven by data. It overcomes the inherent shortcomings of CPUs and GPUs in deep learning by emulating neural networks in hardware, which significantly improves the running speed of deep learning chips and can be used to complete prediction and reasoning tasks. In order to achieving high resource utilization, ML frameworks such as TensorRT Inference Server [5] or TensorFlow Serving [6] provide runtime features for a single NPU to handle multiple DNN inference queries (i.e., multi-tasking DNNs).

While meeting the computing power requirements of embedded systems, it should be noted that embedded real-time systems, such as airborne embedded systems, have timing constraints for applications. In the traditional CPU+GPU architecture, in order to meet the real-time requirements,

978-1-6654-7315-6/23/$31.00 ©2023 IEEE
DOI 10.1109/ICPADS56603.2022.00063



Authorized licensed use limited to: Wuhan University. Downloaded on September 22,2026 at 06:44:38 UTC from IEEE Xplore. Restrictions apply.

some researchers have established various analytical GPU performance models to estimate the application execution time [7] [8] [9] [10] [11] [12] [13]. Moreover, some previous studies have investigated GPU real-time scheduling problem and proposed some preemptive supporting methods [14] [15] [16] [17] [18] [19] [20] [21] [22] [23] [24] [25]. Nevertheless, the NPU does not support preemptive scheduling and the default scheduling policy does not involve the priority of tasks. When a low-priority task requests the NPU, the inherent scheduler will process it immediately and allocate the existing NPU computing resources to the task as much as possible. At this point, if a high-priority task arrives, it will be blocked by the low-priority task. Obviously, this scheduling mechanism cannot meet the real-time requirements of airborne embedded systems. Therefore, it is necessary to enhance the existing task scheduling strategy of NPUs.

In this paper, WMDRS, a workload-aware performance model based multi-task dynamic-quota real-time scheduling for neural processing units is proposed. For a specific application $App_i$, we define workload as the other applications concurrently executing with $App_i$ on a single NPU. The main contributions of this paper are in three aspects:

1) A load-aware performance model that can accurately predict the remaining execution time of each task in multi-task concurrent scenarios is put forward for NPUs.
2) A pseudo-preemptive scheduler is designed to allocate computing resources for each task dynamically. When a high-priority task arrives, the scheduler can achieve approximate preemption by reducing the computing resources held by the low-priority task.
3) A complete software prototype of the WMDRS system is implemented. WMDRS allocates computing resources to each NPU task based on a greedy scheduling strategy. Furthermore, the forementioned NPU performance model and real-time scheduling algorithm are evaluated on a set of realistic tasks. The experimental results show that WMDRS can meet the soft real-time deadline requirement.

The rest of the paper is organized as follows. Section II summarizes the overseas and domestic research status of NPU multi-task real-time scheduling. Section III introduces the complete architecture of WMDRS. Section IV gives the proposed NPU performance model. Section V proposes the pseudo-preemptive real-time scheduling algorithm based on greedy policy. Section VI presents the evaluation results. Section VII concludes the paper.

## II. Related Work

### A. GPU Performance Model

According to our research on the GPU performance model, in order to predict the remaining execution time of applications, various characteristics, such as the number of cores, memory latency, and program abstraction are considered. Nath et al. introduce the pipeline models to revealing the relationship between computation cycles and memory latency [7]. Wang et al. put forward a hybrid framework that combines source code analysis and trace-based simulation to predict the execution time of CUDA kernels [8]. Lew et al. use the simulation method which combines performance counters and specific hardware information to estimate the kernel execution time [9]. Lym et al. design a GPU performance model, DeLTA, especially for the deep learning applications like CNNs. DeLTA accurately models traffic across all memory hierarchy levels [10]. Wu et al. take resources and data transfer cost between CPUs and GPUs into account to predict the execution time of massively parallel programs [11]. Wang et al. establish a novel GPU pipeline performance model with the GPU memory model to predict kernel execution time under different cores and memory frequencies [12].

### B. GPU Preemptive Scheduling

NPUs and GPUs are both designed to provide powerful computing power for applications which conduct a lot of calculation. However, both of them lack the key technology for real-time systems, i.e., preemptive scheduling. If a low-priority application requests the GPU or NPU, the inherent scheduler will process this request in an efficient manner. Usually, the computing resources are allocated to this application as many as possible. When a high-priority application comes, it may be blocked or delayed by the low-priority application, which is known as priority inversion [26], since the default scheduling policy has no regard for the application priority. GPUs employ a SIMT execution model, and thread scheduling is managed in thread-block granularity, upon which previously studied GPU preemption solutions [14], [15] are primarily founded. Previous studies on GPU scheduling can be roughly classified into the following three categories:

1) Scheduling methods based on operating system kernels and device drivers [16] [17] [18];
2) Hardware-based preemption scheduling technology [14] [15] [19];
3) Software-based methods which support kernel preemption, namely kernel slicing technology [20] [21] [22] [23] [24] [25].

### C. NPU Performance and Preemptive Scheduling

As GPU SIMT programming abstraction, the underlying GPU micro-architecture and task scheduling granularity dramatically differ from how NPUs are programmed and executed. It is challenging to apply the scheduling solution of CPUs or GPUs directly to NPU architecture. In order to guarantee timeliness in multi-task concurrent environment, it is necessary to accurately predict timing behavior of an executing application and make further decisions on this basis [13]. In practical scenarios, however, the remaining execution time of any given NPU application is unknown. Therefore, it is challenging to develop a real-time scheduler based on the task deadline. Choi et al. propose a prediction model that estimates network-wide DNN latency for each task [27]. A DNN is expressed as a DAG where each graph node corresponds to a DNN layer. The remaining execution time of one task is



Authorized licensed use limited to: Wuhan University. Downloaded on September 22,2026 at 06:44:38 UTC from IEEE Xplore. Restrictions apply.

estimated by predicting the latency generated by each DNN layer and how many nodes are executed overall. However, not only the prediction model of one application is tightly coupled with the specific network, but also the training of each model needs to consider all of the inference objects, which is an almost impossible job in practice. In addition to the NPU performance model, Choi et al. extend the NPU to including a preemption module, which uses inference task context table to track the specific information of each task [27]. The task queue inside the preemption module is responsible for receiving inference requests from CPUs. They also look at three preemption mechanisms based on CHECKPOINT, KILL and DRAIN, and compare the advantages and disadvantages of the three. At the end of this paper, PREMA, a scheduling policy is presented. When a new task is dispatched from the CPU to the NPU, it is assigned with a fixed, user-defined static priority level, and PREMA decides which preemption mechanism to use. The storage overhead of this method is mainly from the output activations of checkpoints, what is worth mentioning only a single task storage overhead can even reach hundreds of MBs. Rhu et al. [28] solve the problem of NPU local memory oversubscription by copying the overflow data from local memory to CPU memory. During the runtime, when it is observed that NPU memory usage is near its limit, the DMA unit can migrate the checkpointed state from the NPU to CPU memory while the inference is being serviced to hide migration overhead. Unfortunately, the preemption scheduling proposed by them requires an additional on-chip SRAM module to store the context of the preemption task. In addition, it is necessary to note that their research focuses on the low latency requirement of cloud server and does not consider the real-time nature of tasks.

## III. System Architecture

We have implemented the complete architecture of WMDRS, which is shown in Fig. 1 and the whole system consists of the following three modules.

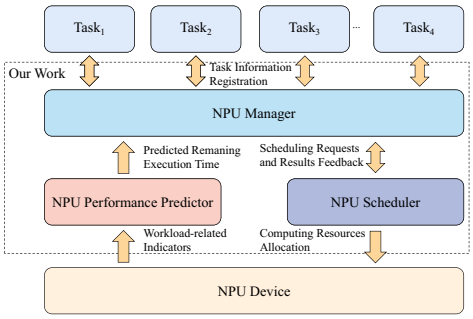



**Fig. 1.** System architecture.

- Resource Manager. The resource manager acts as the overall manager. It is originally designed to sense the arrival of new tasks and the exit of old tasks in real time. The resource manager first abstracts the $App$ into $Task$ and registers it. The registered $Task$ will be completely maintained by the resource manager, including its status and parameters. An important part of registration is the priority of tasks, which will be used as an indispensable scheduling basis. It is worth mentioning that $Task$ is also the unique identifier that the WMDRS system perceives the existence of a task. When an task exits, resource Manager clears all information about the task. In addition, the resource manager also has an important mission, maintaining the global resource status information of the entire system.
- NPU Performance Predictor. NPU Performance Predictor is mainly responsible for collecting real-time workload indicators information of NPU device, which together with specific $Task$ parameter information will be used as input parameters to accurately predict the remaining execution time of each Task through the pre-trained NPU Performance Model.The remaining execution time predicted by NPU Performance model is a key indicator of scheduling.
- NPU Scheduler. NPU Scheduler receives scheduling requests from the resource manager and allocates computing resources to each Task. The NPU resources assigned to the $Task$ will be mapped to the application running on NPU device. We design the NPU Scheduler and implement a pseudo-preemptive scheduling policy. It dynamically adjusts computing resources for active tasks to meet the real-time requirement rather than removes the low-priority task from NPU device at the scheduling point.

## IV. Workload-Aware NPU Performance Model

In this section, we detail our proposed Workload-Aware NPU Performance Model. First, some NPU workload-related indicators are described. Then, we show how to train the BPNN with a given set of tasks.

### A. NPU Workload-Related Indicators

Co-executing tasks on a single NPU lead to resource contention which greatly degrades NPU performance. Different tasks have different requirements on NPU resources, and the execution time of the same task varies in different system states. For a specific $Task_i$, we define the workload as the other tasks concurrently executing with $Task_i$ . Thus, the NPU performance model needs to collect the workload-related indicators as inputs. DSMI (Device System Manage Interface) is used to manage Ascend AI processors. Through this interface, users can obtain information of Ascend AI processors. The DSMI interface supports multi-thread or multi-process invocation in concurrent scenarios. Taking into account the APIs provided by DSMI and the experimental data on Huawei Ascend 310 processor, this paper mainly considers the following four main NPU resources, including $AI\ Core$, $AI\ CPU$, $Ctrl\ CPU$ and $Memory$, which have a significant impact on the execution time of one task. The workload-related indicators are detailed in Table I.



Authorized licensed use limited to: Wuhan University. Downloaded on September 22,2026 at 06:44:38 UTC from IEEE Xplore. Restrictions apply.

TABLE I: The Workload-Related Indicators

| Resources | Indicators |
|---|---|
| AI Core | ai_core_utilization_rate |
| | ai_core_frequency |
| AI CPU | ai_cpu_utilization_rate |
| Ctrl CPU | ctrl_cpu_utilization_rate |
| | ctrl_cpu_frequency |
| Memory | memory_utilization_rate |
| | memory_bw_utilization_rate |
| | memory_frequency |

1) $AI\ Core$. The computing core of Ascend, ai core mainly responsible for matrix, vector, scalar computation intensive operator tasks. We chose two metrics ($ai_core_utilization_rate$ and $ai_core_frequency$). The $ai_core_utilization_rate$ is the real-time utilization rate of $AI\ Core$. The $ai_core_frequency$ is the frequency of $AI\ Core$.
2) $AI\ CPU$. The $AI\ CPU$ responsible for executing operators which are not suitable for running on $AI\ Core$ (undertaking complex non-matrix calculations), selected a metric ($ai_cpu_utilization_rate$). The $ai_cpu_utilization_rate$ is the utilization rate of $AI\ CPU$.
3) $Ctrl\ CPU$. The $Ctrl\ CPU$ is dedicated to controlling the overall operation of the chip, we focus on two indicators ($ctrl_cpu_utilization_rate$ and $ctrl_cpu_frequency$). The $ctrl_cpu_utilization_rate$ clearly means utilization rate of $Ctrl\ CPU$ and the $ctrl_cpu_frequency$ indicates $Ctrl\ CPU$ frequency.
4) $Memory$. For $Memory$, the main storage unit, we choose three metrics ($memory_utilization_rate$, $memory_bw_utilization_rate$ and $memory_frequency$). These three indicators respectively represent the utilization rate of $Memory$, the utilization rate of $Memory$ bandwidth and Memory frequency.

In addition to the metrics mentioned above, the input variables we specified also include $device_temperature$ and the computational force quota assigned to the task (detailed in Section V), which represents the computational resource usage of the task. This indicator is dynamically set by the NPU scheduler rather than the DSMI.

### B. BPNN Based NPU Performance Model

We adopt a Back Propagation Neural Network to model the NPU performance with different workloads. Back Propagation algorithm is one of the most important algorithms in neural network deep learning and BPNN is a multi-layer feed-forward neural network trained by error Back Propagation algorithm. The structure of BPNN for NPU performance estimation is shown in Fig. 2. It is a four-layer network. The first layer is the input layer which contains all workload-related indicators. The second and the third layers are the hidden layers with 13 nervecells respectively. As is shown in Fig. 3, the nerve cell

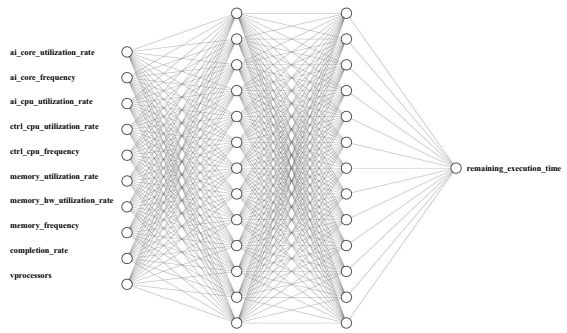



**Fig. 2.** The structure of BPNN.

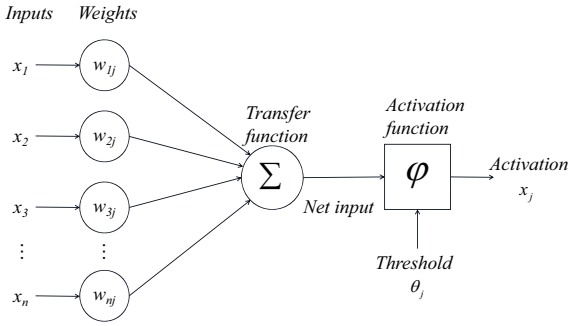



**Fig. 3.** The structure of BPNN.

output value $x_i$ is calculated as follows:

$$S_j = \sum_{i=1}^{n} w_{ij} x_i + \theta_j \tag{1}$$

$$x_j = \frac{1}{1 + e^{-S_i}} \tag{2}$$

where $x_j$ is output of the $j^{th}$ node; $w^{ij}$ is the weight between the $i^{th}$ node and the $j^{th}$ node; $\theta_j$ means threshold of the $j^{th}$ node. Finally, the output is the predicted remaining execution time of task.

### C. Training Data

Usually, the airborne embedded system has a fixed task set, tasks running on NPU are deterministic. Therefore, the BPNN is trained based on a fixed task set $\Gamma$ with $N$ tasks $\{Task_1, Task_2, ..., Task_N\}$. We implement a monitor thread to collect the selected workload-related indicators as input variables for BPNN. In order to train a BPNN for a specific $Task_i$, tasks in set $\Gamma$ except $Task_i$ form different combinations and are co-executing with $Task_i$ on different allocations of computing resources. In order to further obtain more comprehensive training data, we further refine the combination of other tasks in the task set $\Gamma$: For the load tasks, we let them run under different computing force ratios, so as to obtain more comprehensive training data for $Task_i$. The sampling of input and output variables of BPNN is shown in Fig. 4. The system state parameters of the task under different loads are recorded in real time, and the execution time of the task



Authorized licensed use limited to: Wuhan University. Downloaded on September 22,2026 at 06:44:38 UTC from IEEE Xplore. Restrictions apply.

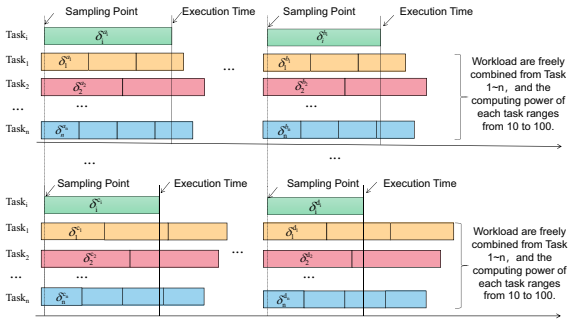



**Fig. 4.** BPNN training data sampling.

is predicted by using the system state parameters and the task calculation force as input values. When training a task model, the other tasks in this set are combined as loads. In a certain load, it is necessary to ensure that the load is constant during the execution of $Task_i$. During this process, we obtain system status parameters by invoking the DSMI interface. The monitor thread samples input vector $X^k$ every 100 milliseconds when $Task_i$ starts. As $Task_i$ is finished, the difference value between each sample timestamp ${t_i}^k$ and the end time ${t_e}^k$ is taken as the remaining execution time of $Task_i$, recorded as $y^k$. In addition, the input parameter also contains a calculated value $F^k$, which is calculated as follows:

$$F^k = \frac{t_i^k - t_s^k}{t_e^k - t_s^k} \tag{3}$$

where ${t_s}^k$ and ${t_e}^k$ indicate the start time and end time of the task respectively. ${t_i}^k$ represents any monitoring thread sampling timestamp during task execution. Given the usage of computing resources, $vprocessors$(detailed in Section V), an input vector $\langle X^k, F^k, \delta \rangle$ of a workload combination is obtained, recording as $X^{k+}$. Then, an input-output pair $\langle X^{k+}, y^k \rangle$ can be obtained naturally. Finally, we can train BPNN when getting enough samples from different workload combinations at various allocations of computing resource.

## V. Pseudo-Preemptive Real-Time Scheduling

In this section, the proposed pseudo-preemptive real-time scheduling algorithm is introduced in detail. First, we present an NPU task model and show how to calculate the remaining time of a task when the workload is changed. Then, the scheduling strategy based on greedy algorithm is described. Finally, we discuss some optimizations on basic implementation to further reduce scheduling overhead.

### A. Deadline-Based Pseudo-Preemptive Scheduling

Considering the case where a high-priority task is co-executing with a low-priority task, unlike previous preemptive NPU scheduling methods that remove a task from an NPU device, the proposed pseudo-preemptive scheduling method dynamically adjusts the number of $vprocessors$ for the active tasks according to priority and deadline. The $vprocessor$ is defined as follows:

$$vprocessor = \frac{1}{period} \tag{4}$$

where $period$ is a fixed time period. This period is not a clock period but a virtual time period.

We consider a task model with constrained dead lines. Specifically, an NPU task is characterized as follows:

$$App_i = \left\langle \tau_i^{\delta,N}, \gamma_i^{\delta,L}, F_i, D_i, T_i, P_i \right\rangle \tag{5}$$

- $\tau_i^{\delta,L}$ is the actual execution time of the task concurrent with the workload $L$ on $\delta$ vprocessors ;
- $\gamma_i^{\delta,L}$ is the predicted remaining execution time when the $Task_i$ runs concurrently with the workload $L$ on on $\delta$ vprocessors;
- ${F_i}^k$ is the completion rate of $Task_i$ when the $k^{th}$ change fo the work-load;
- $D_i$ is the relative deadline of $Task_i$;
- $T_i$ is the period of the $Task_i$;
- $P_i$ is the fixed priority of $Task_i$.

As described in the previous section, the input vector of our proposed NPU performance model includes task completion rate, which requires that the value of this indicator must be calculated in time. Thus, if we record the task completion rate at the time of last load change (0 at the beginning), at each work-load change, we can calculate the current completion rate of $Task_i$ by the completion rate at the time of last load change ${F_i}^{k-1}$, the predicted running time at the time of last scheduling $\gamma_i^{\delta',L_{k-1}}$ and the actual running time $\tau_i^{\delta',L_{k-1}}$ as follow:

$$F_i^k = \frac{\tau_i^{\delta',L_{k-1}}}{\gamma_i^{\delta',L_{k-1}}} \left(1 - F_i^{k-1}\right) + F_i^{k-1} \tag{6}$$

In this way we don't need to keep track of all previous changes of load, just focus on the recent one. More importantly, $F_i$ as one of the input parameters of proposed NPU performance model, the simpler the calculation process, the better. Based on the above formula and proposed NPU performance model, the worst execution time of $Task_i$ on different allocation of $vprocessors$ can be predicted, and then the minimum allocation of $vprocessors$ $\delta'$ can be obtained which meets the real-time requirements. $\delta'$ satisfies the following conditions:

$$\forall \delta \in \left\{ \delta \middle| \tau_i^{\delta,N} \leq \frac{D_i}{1 - F_i} \right\}, \delta' \leq \delta \tag{7}$$

### B. Priority Based Greedy Scheduling Policy

According to above deduction, we implement the pseudo-preemptive real-time scheduling policy based on the greedy algorithm, which is given in Algorithm 1.

All $Task_i \in \Gamma$ are sorted in descending order according to the task priority. The larger the value of $P_i$, the higher the task



Authorized licensed use limited to: Wuhan University. Downloaded on September 22,2026 at 06:44:38 UTC from IEEE Xplore. Restrictions apply.

**Algorithm 1** Priority-Based Gready Scheduling Algorithm

**Input:** $\Gamma$: A Task Set, N: The Number of Applications, j: Change Times of Workload, $F_{i,j}$: The Completion Rate of $Task_i$ When The Workload $j^{th}$ Changes, $D_i$: Relative Deadline of $Task_i$, $P_i$: Priority of $Task_i$, $\tau_i^{\delta_{i,j-1},L_{i,j-1}}$: Actual Exection Time of $Task_i$ with Workload $L_{i,j-1}$ on $\delta_{i,j-1}$ $vprocessors$, $\gamma_i^{\delta_{i,j-1},L_{i,j-1}}$: The Predicted Exection Time of $Task_i$ with Workload $L_{i,j-1}$ on $\delta_{i,j-1}$ $vprocessors$

**Output:** : $\Delta[N]$: $\delta$ Arrary for All Tasks, The Element $\delta_{i,j+1} \in \Delta$ is The Number of $vprocessors$ Allocated to $Task_i$ in The $(j+1)^{th}$ Workload Change

1: $F \leftarrow 0$ , $P \leftarrow 0$ , $Y \leftarrow 0$
2: **for** $Task_i \in \Gamma$ **do**
3:  **if** $Task_i$ is active **then**
4:  $F_{i,j} \leftarrow \frac{\tau_i^{\delta_{i,j},L_{i,j-1}}}{\gamma_i^{\delta_{i,j},L_{i,j-1}}}(1 - F_{i,j-1}) + F_{i,j-1}$
5:  $Y \leftarrow BPNN(\delta_{i,j}, F_{i,j}, L_{i,j})$
6:  $\delta_{i,j+1} \leftarrow FINDMINDELTA(Y)$
7:  **if** $Resource > \delta_{i,j+1}$ **then**
8:  $TOTAL \leftarrow TOTAL - \delta_{i,j+1}$
9:  **end if**
10:  **else**
11:  **return** $\Delta$
12:  **end if**
13: **end for**
14: **for** $Task_i \in \Gamma$ **do**
15:  **if** $Resource > 0$ **then**
16:  $\delta_{i,j+1} \leftarrow min(100 - \delta_{i,j+1}, Resource)$
17:  $\gamma_i^{\delta_{i,j},L_{i,j}} \leftarrow Y[\delta_{i,j+1}]$
18:  $Resource \leftarrow Resource - min(100 - \delta_{i,j+1}, Resource)$
19:  **else**
20:  **return** $\Delta$
21:  **end if**
22: **end for**
23: **return** $\Delta$

priority $(P_i > P_{i+1}, i = \{1, 2, ...N-1\})$ . This greedy policy will prioritize time constraints of high priority applications.

In Algorithm 1, we check all tasks in $\Gamma$. If the $Task_i$ is inactive, the $\delta_{i,j+1}$ is set to 0 which means no $vprocessor$ is allocated to $Task_i$. Otherwise, it should adjust the number of $vprocessors$ allocated for $Task_i$ when workload is changed.

In Step 5, the function ***BPNN*** is to predict remaining execution time of $Task_i$ with the current completion rate $F_{i,j}$, workload-related indicators $L_{i,j}$ and the allocation of $vprocessors$ $\delta_{i,j}$ based on the proposed NPU performance model. Where, $F_{i,j}$ is obtained through Step 4 and the initial value is zero. It is worth noting that the function ***BPNN*** returns a list of predicted results, each of which is a numerical pair consisting of the number of $vprocessors$ and the remaining execution time.

In Step 6, the function ***FINDMINDELTA*** is to find the minimum number of $vprocessors$ on the premise of meeting the real-time requirements.

From Step 7 to Step 11, it is necessary to compare the sum of allocated $vprocessors$ with the entire $vprocessors$ TOTAL.

From Step 13 to Step 20, if there are available $vprocessors$, the system allocates the remaining $vprocessors$ to tasks in priority order. Note that a maximum of 100 $vprocessors$ are allocated to one task. In this way, the computing resources of the system can be fully utilized.

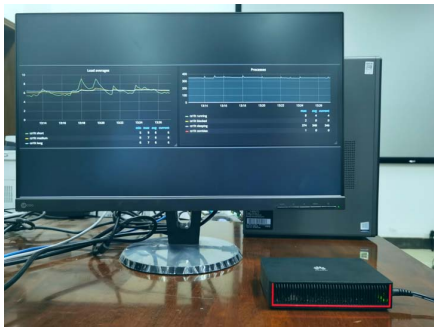


**Fig. 5.** Experiment environment.

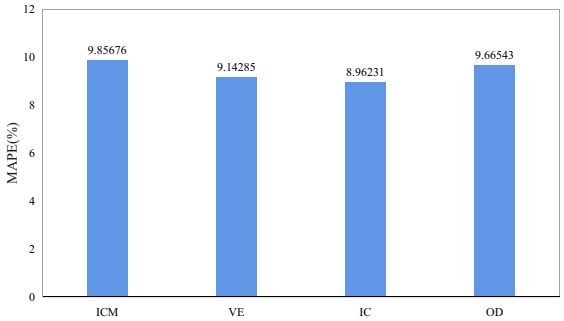



**Fig. 6.** MAPE of the realistic task set.

## VI. Evaluation

### A. Experimental Setup

We implement the prototype of WMDRS on Huawei Atlas 200 DK as shown in Fig. 5.

We evaluate the proposed NPU performance model and scheduling algorithm on a realistic application set. The realistic application set is shown in Table II. It is composed of 4 tasks: Object Detecion(OD), Image color matching(ICM), Image Classification(IC) and Vision Enhancement(VE). All these tasks need to use the NPU device to accelerate performance. In the experiments, we use explicit deadline, which specifies the absolute execution time of the task.

### B. Experiment results

*1) Model Training:* The training phase of $Task_i$ in the task set is conducted as described in Section IV. The workload are composed of other tasks under different quotas and continue



Authorized licensed use limited to: Wuhan University. Downloaded on September 22,2026 at 06:44:38 UTC from IEEE Xplore. Restrictions apply.

TABLE II: Realistic Task Set

| Task | Type | WCET(s) | Relative Deadline | Period(s) | Priority |
|---|---|---|---|---|---|
| Object Detecion(OD) | Periodic | 9.516 | 15 | 20 | 4 |
| Image color matching(ICM) | Periodic | 23.556 | 30 | 40 | 3 |
| Image Classification(IC) | Periodic | 36.240 | 50 | 60 | 2 |
| Vision Enhancement(VE) | Periodic | 56.018 | 80 | 100 | 1 |

to run until $Task_i$ is finished. In this paper, a monitor is implemented to collect the input parameter values of ***BPNN***. The task executing time is calculated based on computer, and all tasks keep running periodically.

During this process, the monitor samples every 100 milliseconds so that the remaining execution time and the completion rate of the $Task_i$ can be calculated based on the timestamp and the actual completion time of the task. Then the sampled data, the number of $vprocessors$ and the calculated completion rate are taken as the input of BPNN. The calculated remaining execution time of $Task_i$ is taken as the output of ***BPNN***. 4000 times are run for one task, and 300,000 sample pairs $\langle X^{k+}, y^k \rangle$ are collected. The BPNN model for each task is trained by 250,000 sample pairs, and the remaining samples are used to evaluate the accuracy of workload-aware NPU performance model.

*2) Overall Accuracy:* We choose the Mean Absolute Percentage Error(MAPE) as the accuracy metric which is defined as follows:

$$MAPE = \frac{1}{M}\sum_{k=1}^{M}\frac{|S_m^k - S_e^k|}{S_e^k} \times 100\% \tag{8}$$

where $S_m^k$ is the measured execution time via the monitor thread and $S_e^k$ is the estimate remaining execution time via the BPNN model.

In the evaluation, the workload still remains stable with the task being tested. From Fig. 6, the proposed NPU performance model can achieve alow average MAPE from 8.96231% to 9.85676%.

*3) Schedulability:* Fig. 7 compares the schedulability ratio of application set as a function of the number of tasks between the inherent Ascend scheduler and the proposed WMDRS scheduler. It is obviously that the schedulability ratio decreases with the increasing number of tasks, because the resource contention becomes more intense with co-executing applications.

In order to analyze the merits and demerits of WMDRS scheduling results, we record the start time and completion time series of all tasks through the designed monitor, as shown in Fig. 8. We made statistics on the execution of tasks scheduled by WMDRS and Ascend scheduler respectively for 2 hours. The results show that under Ascend scheduler, the timeout ratio of ICM and OD is 28% and 54.198% respectively. In the case of WMDRS scheduling, none of the tasks timed out, and the success rate is 100%. On four long test runs (30, 60, 90, and 120 minutes respectively), the scheduling success rate has always been 100%.

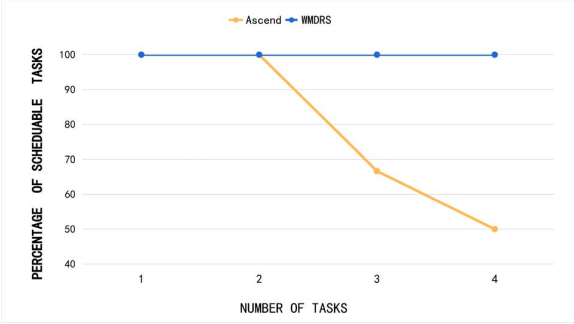



**Fig. 7.** Schedulability versus the number of tasks.

*4) Scheduling Overhead:* WMDRS is different from previous NPU scheduling methods that save preemption point context. It does not need any hardware extension, but dynamically adjusts the number of $vprocessors$ to achieve approximate preemption. For a task set composed of four tasks, with almost no memory overhead, the scheduling overhead is less than 300 microseconds. Since NPU is still performing inference tasks during the scheduling process, this effect will be smaller than that of numerical display above.

## VII. Conclusion

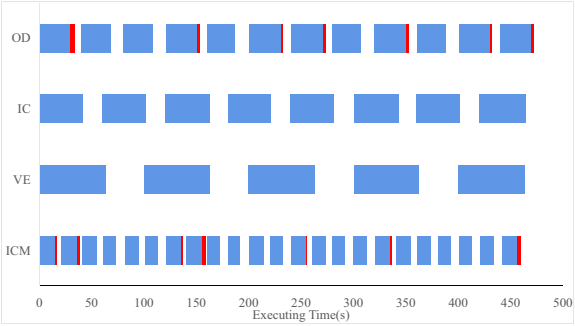
In this paper, we implement a real-time NPU scheduler, WMDRS, for the airborne embedded system. First, in order to predict the remaining execution time of tasks under co-execution circumstance, we propose a workload-aware NPU performance model which collects workload-related indicators at runtime and estimates the remaining execution time of the task based on BPNN. Then, a pseudo-preemptive NPU scheduler is designed to deal with the real-time scheduling in the embedded system. The pseudo-preemptive scheduler dynamically adjusts computing resources for active applications according to the priority and deadline based on greedy policy. Finally, we implement a prototype of WMDRS on Atlas 200 DK and evaluate the proposed model and algorithm on realistic sets. Experimental results illustrate that the proposed WMDRS scheduler can achieve an average predicted error of 8.96231%-9.85676% and the scheduling success ratio has always been 100% in four experiments of different duration.

## Acknowledgment

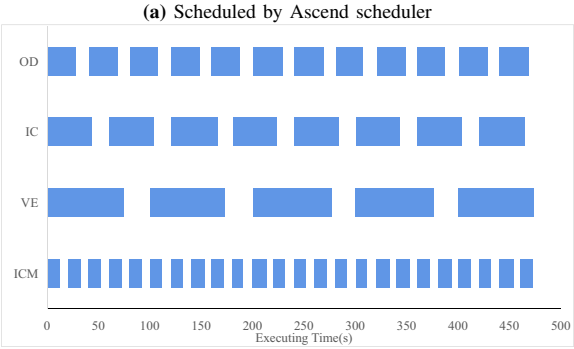
This work was supported in part by the National Natural Science Foundation of China (No. 61876151, 62032018, 62141220) and the Fundamental Research Funds for the Central Universities (No. 3102019DX1005).



Authorized licensed use limited to: Wuhan University. Downloaded on September 22,2026 at 06:44:38 UTC from IEEE Xplore. Restrictions apply.




**(a)** Scheduled by Ascend scheduler




**(b)** Scheduled by WMDRS

**Fig. 8.** Task execution sequence diagram.

## REFERENCES


[1] Y. -H. Chen, J. Emer and V. Sze, "Eyeriss: A Spatial Architecture for Energy-Efficient Dataflow for Convolutional Neural Networks," 2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA), 2016, pp. 367-379.

[2] J. Albericio, P. Judd, T. Hetherington, T. Aamodt, N. E. Jerger and A. Moshovos, "Cnvlutin: Ineffectual-Neuron-Free Deep Neural Network Computing," 2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA), 2016, pp. 1-13.

[3] A. Parashar et al., "SCNN: An accelerator for compressed-sparse convolutional neural networks," 2017 ACM/IEEE 44th Annual International Symposium on Computer Architecture (ISCA), 2017, pp. 27-40.

[4] N. P. Jouppi et al., "In-datacenter performance analysis of a tensor processing unit," 2017 ACM/IEEE 44th Annual International Symposium on Computer Architecture (ISCA), 2017, pp. 1-12.

[5] NVIDIA, "TensorRT Inference Server User Guide," 2018.

[6] Google, "TensorFlow Serving for Model Deployment in Production," 2018.

[7] R. Nath and D. Tullsen, "The CRISP performance model for dynamic voltage and frequency scaling in a GPGPU," in Proc. 48th Annu. IEEE/ACM Int. Symp. Microarchit., 2015, pp. 281-293.

[8] X. Wang, K. Huang, A. Knoll, and X. Qian, "A hybrid framework for fast and accurate GPU performance estimation through source-level analysis and trace-based simulation," in Proc. IEEE Int. Symp. High Perform. Comput. Archit., 2019, pp. 506-518.

[9] J. Lew et al., "Analyzing machine learning workloads using a detailed GPU simulator," in Proc. IEEE Int. Symp. Perform. Anal. Syst. Softw., 2019, pp. 151-152.

[10] S. Lym, D. Lee, M. O' Connor, N. Chatterjee, and M. Erez,"DeLTA: GPU performance model for deep learning tasks with in-depth memory system traffific analysis," in Proc. IEEE Int. Symp. Perform. Anal. Syst. Softw., 2019, pp. 293-303.

[11] J. Wu, X. Yang, Z. Zhang, G. Chen, and R. Mao, "A performance model for GPU architectures that considers on-chip resources: Task to medical image registration," IEEE Trans. Parallel Distrib. Syst., vol. 30, no. 9, pp. 1947-1961, Sep. 2019.

[12] Q. Wang and X. Chu, "GPGPU performance estimation with core and memory frequency scaling," IEEE Trans. Parallel Distrib. Syst., vol. 31, no. 12, pp. 2865-2881, Dec. 2020.

[13] Yuan Yao, Shuangyang Liu, Sikai Wu, Jinyu Wang, Jinting Ni, Gang Yang and Yu Zhang, "$WAMP^2S$: Workload-Aware GPU Performance Model Based Pseudo-Preemptive Real-Time Scheduling for the Airborne Embedded System," IEEE Transactions on Parallel and Distributed Systems, 2022, vol. 33, no. 11, pp. 2767-2780.

[14] I. Tanasic, I. Gelado, J. Cabezas, A. Ramirez, N. Navarro, and M. Valero, "Enabling Preemptive Multiprogramming on GPUs," in Proceedings of the International Symposium on Computer Architecture (ISCA), 2014.

[15] J. Park, Y. Park, and S. Mahlke, "Chimera: Collaborative Preemption for Multitasking on a Shared GPU," in Proceedings of the International Conference on Architectural Support for Programming Languages and Operation Systems (ASPLOS), 2015.

[16] S. Kato, K. Lakshmanan, A. Kumar,M. Kelkar, Y. Ishikawa, and R. Rajkumar, "RGEM: A responsive GPGPU execution model for runtime engines," in Proc. 32nd IEEE Real-Time Syst. Symp., 2011, pp. 57-66.

[17] S. Kato, M. McThrow, C. Maltzahn, and S. A. Brandt, "Gdev: First class GPU resource management in the operating system," in Proc. USENIX Annu. Tech. Conf., 2012, pp. 401-412.

[18] G. A. Elliott, B. C. Ward, and J. H. Anderson, "GPUSync: A framework for real-time GPU management," in Proc. 34th IEEE RealTime Syst. Symp., 2013, pp. 33-44.

[19] Z. Wang, J. Yang, R. Melhem, B. Childers, Y. Zhang, and M. Guo, "Simultaneous multikernel GPU: Multi-tasking throughput processors via fifine-grained sharing," in Proc. IEEE Int. Symp. High Perform. Comput. Archit., 2016, pp. 358-369.

[20] J. Zhong and B. He, "Kernelet: High-throughput GPU kernel executions with dynamic slicing and scheduling," IEEE Trans. Parallel Distrib. Syst., vol. 25, no. 6, pp. 1522-1532, Jun. 2014.

[21] H. Zhou, G. Tong, and C. Liu, "GPES: A preemptive execution system for GPGPU computing," in Proc. 21st IEEE Real-Time Embedded Technol. Appl. Symp., 2015, pp. 87-97.

[22] G. Chen, Y. Zhao, X. Shen, and H. Zhou, "EffifiSha: A software framework for enabling efffificient preemptive scheduling of GPU," in Proc. 22nd ACM SIGPLAN Symp. Princ. Pract. Parallel Program., 2017, pp. 3-16.

[23] B. Wu, X. Liu, X. Zhou, and C. Jiang, "FLEP: Enabling flflexible and effificient preemption on GPUs," in Proc. 22nd Int. Conf. Architectural Support Program. Lang. Oper. Syst., 2017, pp. 483-496.

[24] C. Hartmann, U. Margull, "GPUart-An task-based limited preemptive GPU real-time scheduler for embedded systems," J. Syst. Archit., vol. 97, pp. 304-319, 2019.

[25] H. Lee, H. Kim, C. Kim, H. Han, and E. Seo, "Idempotence-based preemptive GPU Kernel scheduling for embedded systems," IEEE Trans. Parallel Distrib. Syst., vol. 70, no. 3, pp. 332-346, Mar. 2021.

[26] G. A. Elliott and J. H. Anderson, "Real-world constraints of GPUs in real-time systems," in Proc. 17th IEEE Int. Conf. Embedded RealTime Comput. Syst. Appl., 2011, pp. 48-54.

[27] Y. Choi and M. Rhu, "PREMA: A Predictive Multi-Task Scheduling Algorithm For Preemptible Neural Processing Units," High-Performance Computer Architecture IEEE, 2020.

[28] M. Rhu, N. Gimelshein, J. Clemons, A. Zulfifiqar, and S. W. Keckler, "vDNN: Virtualized Deep Neural Networks for Scalable, Memory-Effificient Neural Network Design," in Proceedings of the International Symposium on Microarchitecture (MICRO), 2016.




Authorized licensed use limited to: Wuhan University. Downloaded on September 22,2026 at 06:44:38 UTC from IEEE Xplore. Restrictions apply.